Robotic Planning and Multi-Agent Systems

NECKER

July 9, 2026

Contents

1 Foundations of Robotic Planning

Planning is formalized as the problem of selecting a sequence of actions to reach a designated goal. This domain represents the intersection where Artificial Intelligence converges with Robotics, blurring the boundaries between physical robots and software agents.

- **Deterministic Planning (Open-Loop):** This approach assumes that the robot has perfect and immutable knowledge of the environment and that there are no unexpected events or random disturbances during movement. The mathematical objective is to compute in advance an exact sequence of control actions $\mathcal{U} = \{u_1, u_2, ..., u_n\}$ to guide the system along an ideal trajectory, defined by a series of states $x_1, x_2, \ldots, x_n$, ensuring that the final state coincides with the goal ($x_n \in X_g$). The mechanism is based on a discrete-time deterministic transition function:

  $$x_{k+1} = f(x_k, u_{k+1}) \tag{1}$$

  In simple words, if we know the current state and the action we are about to perform, the subsequent state is already written, completely predictable and free of uncertainties.

  During this phase, the surrounding space and known static obstacles are mapped and discretized into geometric structures such as topological graphs or occupancy grids. The search for the optimal route is then assigned to classical algorithms (such as A^*, Dijkstra, or incremental planners), which extract the most efficient sequence of steps under purely geometric constraints, momentarily ignoring the physical details and perturbations of real movement.

- **Feedback Planning and Policy Control (Closed-Loop):** In dynamic scenarios, blindly following a rigid sequence planned in an open loop almost always leads to failure. *Feedback Planning* overcomes this limitation by introducing the concept of a policy $\pi : X \to U$. Instead of computing a fixed list of actions, the robot learns a decision rule that tells it exactly what to do in real-time for *any* situation or state $x \in X$ it might find itself in: $u = \pi(x)$. This approach assumes that the system is capable of constantly monitoring its position and orientation using onboard sensors.

  When the effects of actions are not certain but probabilistic, the problem is rigorously formalized through **Markov Decision Processes (MDPs)**, defined by the tuple $\langle X, U, P, R, \gamma \rangle$. In this case, the uncertainty of the physical world is captured by the transition probability:

  $$P(x_{k+1} \mid x_k, u_k) \tag{2}$$

  which defines how likely it is to end up in a specific next state starting from the current one and applying a specific action. The robot's behavior is guided by an immediate reward function $R(x_k, u_k)$ (which rewards correct behaviors and penalizes errors) and a discount factor $\gamma \in [0, 1)$ that weighs the importance of future results compared to immediate ones.

2 Heuristic Search and the A^* Algorithm

The A^* algorithm is an informed *best-first* search technique, widely used to find the minimum-cost path between a starting node and a goal node within a graph. Unlike blind search algorithms, A^* leverages domain-specific knowledge to guide the exploration, expanding nodes in a way that minimizes the following cost function:

$$f(s) = g(s) + h(s) \tag{3}$$

where:

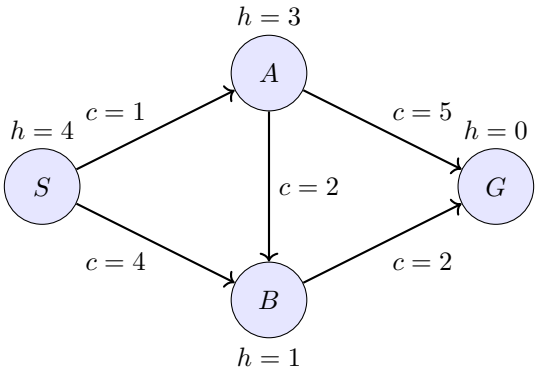
- $g(s)$ is the actual accumulated cost to reach state s from the initial node.
- $h(s)$ is the **heuristic**, which is an estimate of the remaining cost required to reach the goal starting from s.

Algorithm Mechanics

The search process generally maintains two main data structures: an *Open List* (or frontier), which contains discovered but not yet expanded nodes, and a *Closed List* (EXL), which tracks already analyzed nodes.

The algorithm proceeds iteratively according to these steps:

1. Extracts the node with the minimum $f(s)$ value from the Open List.
2. If this node corresponds to the goal state, the algorithm terminates and reconstructs the optimal path backwards.
3. Otherwise, the node is moved to the Closed List and "expanded": its successor nodes are generated.
4. For each successor, the new values of g, h (depending on the problem), and f are calculated. If the successor is not in either list, it is added to the Open List. If it is already present but the new path offers a lower g cost, its value is updated.



- **Initialization:** The starting node S is inserted into the Open List.
  - $g(S) = 0$, $h(S) = 4 \implies f(S) = 0 + 4 = 4$.
  - **Open List:** $\{S(f = 4)\}$
  - **Closed List:** $\{\}$
- **Step 1: Expansion of** S The node with the lowest f is extracted from the Open List, which is S. Since $S \neq G$, it is inserted into the Closed List and its successors, A and B, are generated.
  - For A: $g(A) = g(S) + c(S, A) = 0 + 1 = 1$. Since $h(A) = 3$, we have $f(A) = 1 + 3 = 4$.
  - For B: $g(B) = g(S) + c(S, B) = 0 + 4 = 4$. Since $h(B) = 1$, we have $f(B) = 4 + 1 = 5$.
  - **Open List:** $\{A(f = 4), B(f = 5)\}$ (sorted by f)
  - **Closed List:** $\{S\}$

- **Step 2: Expansion of** A The node with the lowest f is extracted from the Open List, which is A. A is moved to the Closed List and its neighbors, B and G, are evaluated.
  - For G: $g(G) = g(A) + c(A, G) = 1 + 5 = 6$. Since $h(G) = 0$, we have $f(G) = 6 + 0 = 6$.
  - For B: the path passing through A is evaluated. The new cost would be $g_{new}(B) = g(A) + c(A, B) = 1 + 2 = 3$. Since this cost (3) is lower than the previously calculated $g(B)$ cost (4), the value of B is updated in the Open List: $g(B) = 3 \implies f(B) = 3 + 1 = 4$.
  - **Open List:** $\{B(f = 4), G(f = 6)\}$
  - **Closed List:** $\{S, A\}$
- **Step 3: Expansion of** B The node with the lowest f is extracted, which is B. B is moved to the Closed List and its successors are generated. The only successor not yet in the Closed List is G.
  - For G: the path passing through B is evaluated. The new cost would be $g_{new}(G) = g(B) + c(B, G) = 3 + 2 = 5$. Since this cost (5) is lower than the previous $g(G)$ cost (6), the value of the goal node is updated in the Open List: $g(G) = 5 \implies f(G) = 5 + 0 = 5$. (if it had cost more, it simply wouldn't update in the list and the previous value would be kept)
  - **Open List:** $\{G(f = 5)\}$
  - **Closed List:** $\{S, A, B\}$
- **Step 4: Extraction of** G **(Termination)** The node with the lowest f is extracted from the Open List, which in this case is the goal G itself.
  - Since the extracted node is the *goal*, the algorithm terminates successfully.
  - The optimal path is reconstructed backwards by tracing parent pointers: $S \to A \to B \to G$, with a total actual cost equal to $g(G) = 5$.

2.1 Bounded Suboptimal Search: The A^*_ϵ Algorithm and FOCAL Search

In real-world applications, searching for a perfectly optimal path using classical A^* can demand unsustainable amounts of time and memory, as the algorithm is forced to explore all nodes that could theoretically offer a lower cost. To overcome this limitation, the concept of **bounded suboptimal search** is introduced.

The objective is no longer finding the absolute minimum cost C^*, but guaranteeing the generation of a solution whose total cost C is mathematically bounded by a user-selected tolerance factor $\epsilon \geq 0$:

$$C \leq (1 + \epsilon) \cdot C^* \tag{4}$$

The pioneer of this approach is the A^*_ϵ algorithm (or *Focal Search*), which relaxes the constraint of strictly expanding optimal nodes (OPT) to drastically accelerate exploration.

The Structure: OPEN List and FOCAL List

While classical A^* selects nodes based exclusively on the minimum value of the function $f(s) = g(s) + h(s)$, *Focal Search* splits frontier management by introducing a second dynamic list:

1. ***OPEN List*:** Maintains the same function as the classical case, containing all generated and not yet expanded nodes, sorted by increasing values of $f(s)$. Let f_{min} be the lowest f value currently present in this list:
$$f_{min} = \min_{s \in \text{OPEN}} f(s) \tag{5}$$
The value f_{min} represents the current lower bound on the cost of the optimal solution.
2. ***FOCAL List*:** Is a rolling subset of the *OPEN List*. It contains only those nodes whose estimated cost $f(s)$ does not exceed f_{min} by more than the established tolerance factor:
$$\text{FOCAL} = \{s \in \text{OPEN} \mid f(s) \leq (1 + \epsilon) \cdot f_{min}\} \tag{6}$$

Expansion Strategy and Relationship with OPT

Nodes belonging to the set of optimal paths (OPT) are implicitly protected by the lower bound f_{min}. As long as a node is inside the *FOCAL List*, the algorithm has mathematical certainty that expanding it will not violate the suboptimality bound $(1+\epsilon)$.

The true power of the algorithm lies in the selection criterion within the $FOCAL$ list:

- Instead of extracting the node with the lowest $f(s)$ value, the algorithm sorts and extracts from the $FOCAL$ list the node that minimizes a **second independent heuristic function**, often denoted as $h_F(s)$ or $d(s)$.

The function $d(s)$ typically estimates the **remaining number of steps (edges)** missing to reach the goal, completely ignoring their monetary or geometric cost.

Advantages of the Approach

In this way, the algorithm behaves extremely efficiently:

- **Speed:** By choosing the closest node in terms of steps to the goal (via $d(s)$), the algorithm "points" straight to the *goal* almost like a *greedy* search, reducing the number of expanded nodes by up to several orders of magnitude.
- —Formal Guarantee:— The *OPEN List* and the constraint imposed on the $FOCAL$ list act as a safety barrier. Should the secondary heuristic push the algorithm toward excessively costly nodes, they would exceed the threshold $(1+\epsilon)\cdot f_{min}$ and be automatically evicted from the $FOCAL$ list, forcing the algorithm back toward nodes close to optimal ones (OPT).

Properties of the Heuristic

To guarantee the optimality of the discovered solution, the heuristic $h(s)$ must satisfy precise mathematical properties:

- **Admissibility:** A heuristic is admissible if it never overestimates the cost of the minimum path toward the goal. In practical terms, it must be strictly optimistic: for each node s, $h(s) \leq h^*(s)$, where $h^*(s)$ is the actual remaining cost.

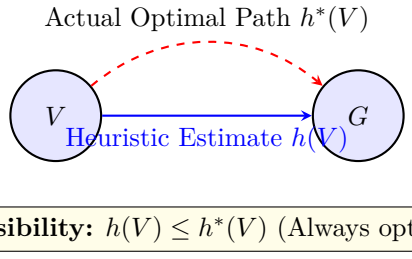


Figure 1: Admissibility Property: The heuristic $h(V)$ always underestimates the actual cost $h^*(V)$ required to reach the goal G.

- **Consistency (or Monotonicity):** This is a stricter requirement, fundamental for searches that use closed lists (EXL). Given two states V and U connected by an action a, the heuristic h is consistent if it satisfies the triangle inequality:

$$h(V) \leq c(V, a, U) + h(U) \tag{7}$$

where $c(V, a, U)$ is the exact cost to transition from V to U. A consistent heuristic is always implicitly admissible. If a closed list (EXL) is used with a heuristic that is *only* admissible but not consistent, the algorithm does not guarantee optimality unless already explored nodes are re-inserted into the Open List when a better path to reach them is found (a highly expensive operation).

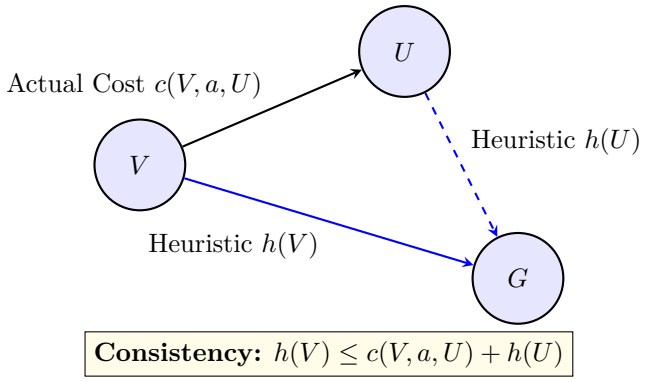


Figure 2: Consistency Property: The heuristic must respect the triangle inequality between two neighboring nodes and the Goal.

3 Dynamic Task Assignment: MAPD

The **Multi-Agent Pickup and Delivery (MAPD)** problem models real-world scenarios where a fleet of agents (for instance, robots in a logistics warehouse) must manage a continuous stream of requests. Unlike classical planning where all tasks are known a priori, in MAPD tasks τ_i arrive dynamically at *runtime*.

Each task τ_i is characterized by two fundamental parameters:

- s_i: the *pickup* vertex (origin location).
- g_i: the *delivery* vertex (destination location).

A task is considered successfully completed when an assigned agent travels from its current position to s_i, picks up the item, and subsequently navigates toward g_i to deliver it, all without colliding with other agents.

The performance of the MAPD system is evaluated primarily through two metrics:

1. **Makespan:** The total number of time *steps* required for the last task of the entire system to be completed.
2. **Service time:** The average waiting time of a task, measured from the instant it appears in the system to the moment it is actually assigned and picked up.

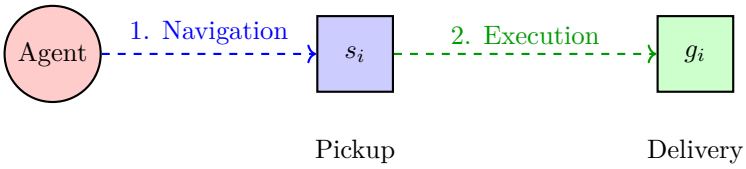


Figure 3: Basic lifecycle of a task in the MAPD problem: the unassigned agent travels to the pickup node and then heads to the delivery node.

3.1 The Token Passing (TP) Algorithm

To solve MAPD in highly dynamic environments, a standard approach is the **Token Passing (TP)** algorithm. It is an —online— and *decentralized* algorithm.

The core of the system is a shared "Token", which is a data structure containing the paths currently planned by all agents and the list of unassigned tasks. When an agent becomes free (has completed its previous task), it requests the Token. Once obtained, the agent chooses a new task from the list, plans a collision-free path (respecting the future paths already reserved by other agents in the Token), and finally passes the Token to the next agent (the token represents write permission, which prevents two agents from concurrently writing two colliding paths while falsely assuming they do not collide with anyone).

If **there are no eligible tasks** (the queue is empty), a free agent does not remain stationary where it is, as it would risk blocking the paths of others. The TP algorithm dictates that the agent heads toward a *safe location* (a secure spot, such as a parking space or a charging station) by planning a collision-free path and enters a waiting state.

3.2 Predictive Variants: Anticipating the Future

The basic TP algorithm only reacts to already existing tasks. However, in many real-world scenarios, task arrivals follow predictable probabilistic models. To slash *Service time*, predictive variants have been developed that allow agents to move strategically even before a task is generated.

- **TP-m2 (Basic Speculation):** This variant introduces the concept of *speculation*. If there are no current tasks, instead of heading to a generic *safe location*, the agent computes a score for various "candidate" vertices where a task is likely to appear shortly. The score S for a candidate s is calculated as:

$$S = \frac{\sum P(t, s)}{1 + h(v, s)} \tag{8}$$

  where the numerator represents the aggregated probability P that a task appears at vertex s at time step t, while $h(v, s)$ is the heuristic distance (the travel cost) from the agent's current position v to the candidate s. The agent will head toward the candidate vertex that maximizes S, thus balancing high probability with spatial proximity.

- **TP-m1 (Speculative Priority):** Goes a step further compared to TP-m2. In TP-m1, an agent evaluates whether it is more convenient to take on a task that *currently exists* but is very far away, or ignore it and speculate on a *future* task that has a very high probability of appearing close to it. This approach drastically reduces idle times but introduces the risk of *starvation* (distant and isolated tasks that are never served). To mitigate this issue, the system can force assignment by temporarily blocking the generation of new tasks until old ones are cleared.

- **Preemption:** To make speculation even more effective, preemption logic is introduced. When an agent decides to speculate toward a candidate vertex, it gains "preemption rights" over a geographical area (a *preemption zone* of radius d) surrounding the target vertex. If a new task appears in this zone while the agent is en route, the agent instantly auto-assigns it, aborting the speculative journey to immediately begin executing the task. This saves precious reassignment time and bypasses the need to request the token (it writes its trajectory directly into memory since it is extremely close to the objective and it is unlikely that another agent's path will collide).

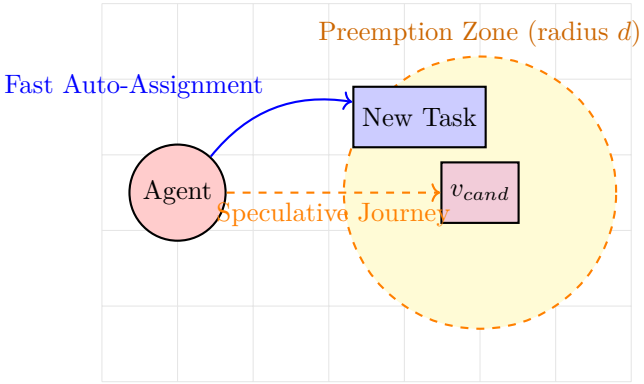


Figure 4: Preemption Logic: the agent is traveling toward a candidate node v_{cand}. As soon as a new task appears inside its preemption zone (radius d), the agent detours and auto-assigns it immediately.

4 Multi-Agent Planning (MAPF and MRPP)

Multi-Robot Path Planning (MRPP) is the problem that requires determining safe and efficient paths to complete distributed tasks across multiple robotic units, often accounting for physical and kinematic constraints. Its formal and algorithmic generalization is named **Multi-Agent Path Finding (MAPF)**.

In abstract MAPF, the goal is to find optimal, collision-free (*non-interfering*) paths for a set of k agents. The problem is typically modeled on a graph $G = (V, E)$, where each agent $i \in \{1, \ldots, k\}$ possesses a starting vertex s_i and a goal vertex g_i. Time is discretized, and at each time step, an agent can move to an adjacent vertex or wait at its current vertex.

4.1 Conflict Types and Complexity

To ensure that paths are safe, a MAPF solution must feature no conflicts. The two fundamental conflicts are:

- **Vertex Conflict:** Two agents attempt to occupy the same vertex v at the same time step t.
- **Edge Conflict:** Two agents attempt to traverse the same edge (u, v) in opposite directions at the same time step, "swapping" places.

The problem generally requires minimizing a global cost function, such as the sum of individual path durations (*Sum of Costs*) or the arrival time of the last agent (*Makespan*). Finding an optimal solution for MAPF is classified as an NP-Hard problem, as the state space grows exponentially with the number of agents.

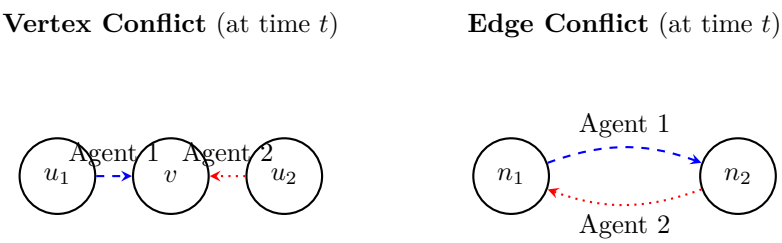


Figure 5: Representation of the two fundamental constraints in MAPF. Left: collision on a shared vertex. Right: head-on collision on an edge.

4.2 The Conflict-Based Search (CBS) Algorithm

Currently, the state-of-the-art for exact MAPF solutions is **Conflict-Based Search (CBS)**. CBS is a complete and optimal algorithm operating on a two-level hierarchical architecture, decoupling global search from single-agent planning.

- **Low level (Single Agent Search):** At this level, the algorithm computes the optimal path for a *single* agent from its start node to its goal, respecting a list of space-time constraints imposed upon it. This is usually solved with a space-time variant of A^*.
- **High level (Constraint Tree):** The high level performs a *best-first* search on a Constraint Tree (CT). Each CT node contains:
  1. A set of constraints (space-time prohibitions such as: agent i cannot be at v at time t).
  2. A solution (a set of paths for all agents satisfying the node's constraints).
  3. The total cost of the solution.

Example:

- **CT Root Node (No initial constraints):** The high level starts with an empty constraint set. It requests the low level (Space-Time A^*) to compute individual shortest paths for each agent, ignoring the others.
  - **Low-level:** A_1 plans path $[S_1 \to B \to C \to G_1]$. It is at C at time $t = 2$.
  - —Low-level:— A_2 plans path $[S_2 \to D \to C \to G_2]$. It is at C at time $t = 2$.
  - **High-level (Validation):** The high level examines the paths and finds a **conflict**: $\langle A_1, A_2, C, t = 2\rangle$. Both agents collide in cell C at time 2.

  The high level then decides to **add two branches** to the tree (two child nodes), attempting to resolve the conflict in two opposing ways.

- **Left Child Node (Prohibited for A_1):** The high level imposes the constraint: *"Agent A_1 cannot be at C at time $t = 2$"*. This restriction does not apply to A_2.
  - **Low-level:** The low level recomputes the path **only for** A_1, since nothing changed for A_2. The old path of A_1 is no longer valid because it violates the constraint.
  - **New plan for A_1:** Space-time A^* finds a detour or makes A_1 wait for one step. The new path is $[S_1 \to B \to B \to C \to G_1]$ (cost increased by 1, passes through C at time $t = 3$).
  - **High-level (Validation):** The high level re-checks all overall paths. Now A_2 passes through C at time 2, while A_1 passes through it at time 3. **No conflicts remaining**. This node becomes a valid and optimal solution.

- **Right Child Node (Prohibited for A_2):** In parallel (being a best-first search), the high level also explores the alternative: *"Agent A_2 cannot be at C at time $t = 2$"*.
  - **Low-level:** This time, A_1's path remains the original one. **Only A_2's path is recomputed**, forcing it to avoid C at time 2.
  - **New plan for A_2:** A_2 finds an alternative route by bypassing C.
  - **High-level (Validation):** This branch is also validated. If there are no conflicts, the high level will compare its cost with that of the left branch and choose the global path minimizing the sum of costs for all agents.

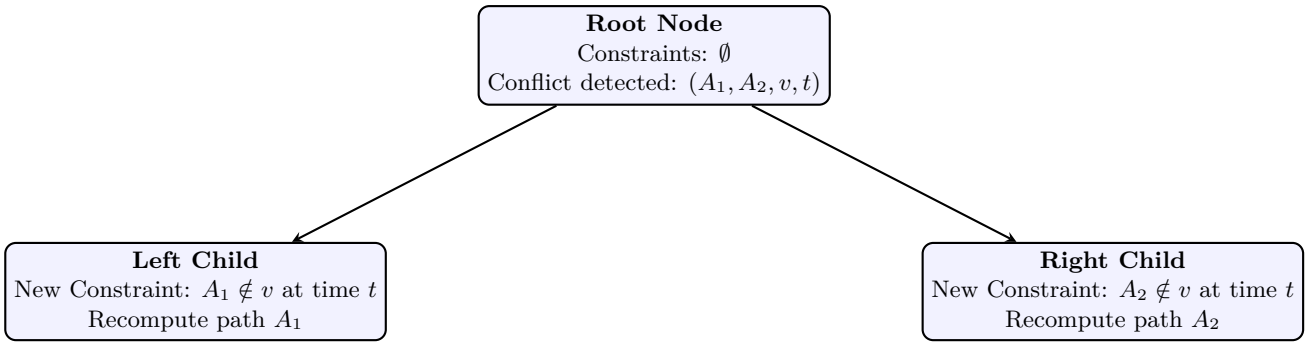


Figure 6: Split process in the constraint tree (High Level) of CBS. Resolving a conflict between two agents A_1 and A_2 generates two parallel scenarios to explore.

4.3 Extensions: Configurable MAPF (C-MAPF)

A natural evolution tied to modern logistics systems (such as Amazon Robotics automated warehouses) is **C-MAPF (Configurable MAPF)**. While in traditional MAPF the environment is static, C-MAPF introduces an additional dimensional complexity by allowing agents to dynamically modify the graph itself (for example, carrying shelves or obstacles from one vertex to another). This configurability, although vastly increasing the *branching factor* of the problem, enables joint optimization of both robot paths and the temporary layout of the workspace.

5 Limitations of Foundational Models (LLMs) in Planning

In recent years, the rise of *Large Language Models* (LLMs) has raised questions about their true capacity to act as planning agents. Although these models showcase complex emergent patterns and remarkable linguistic fluidity thanks to *Attention* mechanisms, they systematically fail in actual deterministic planning.

As highlighted by researcher Subbarao Kambhampati, the fundamental error lies in anthropomorphizing the model's outputs: one must not mistake sequential token generation (a statistical prediction) for true logical "traces of reasoning". LLMs function as autoregressive function approximators: they can "mimic" the statistical distribution of a trace produced by a formal algorithm (like A^*) if they have seen enough of it in their training data, but **they do not possess an internal causal representation** of the world. Planning demands combinatorial exploration, state validation, *lookahead* (predicting consequences) and, crucially, formal *backtracking* mechanisms to recover from errors (as done by A^*'s *Open List*). Because LLMs generate the solution in a single predictive pass (lacking an integrated graph solver), they often produce plans that appear linguistically plausible but, in practice, violate physical laws or logical preconditions of the problem (planning hallucinations).

6 Motion Planning and Configuration Space

To employ search algorithms within physical space, it is necessary to map the **Workspace** $\mathcal{W}$ (the real world where the robot lives, typically in $\mathbb{R}^2$ or $\mathbb{R}^3$) into the **Configuration Space (C-Space)** $\mathcal{C}$.

In $\mathcal{C}$-Space, the robot loses its geometric shape and collapses into a single mathematical **point**. The coordinates of this point uniquely describe all *Degrees of Freedom* (DOF) of the robot. For instance, a flat differential robot in $\mathcal{W} \in \mathbb{R}^2$ has 3 DOFs: position (x, y) and orientation (θ). Therefore, its $\mathcal{C}$-Space will be three-dimensional.

Obstacles and Constraints in $\mathcal{C}$-Space

To compensate for reducing the robot to a single point, real-world obstacles are geometrically "inflated" (via an operation known as Minkowski Sum) based on the shape and radius of the robot. The $\mathcal{C}$-Space is thus split into two disjoint sets:

- **Obstacle Space $\mathcal{O}$:** All configurations where the robot would collide with the environment or with itself.
- **Free Space $\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{O}$:** The set of all safe configurations.

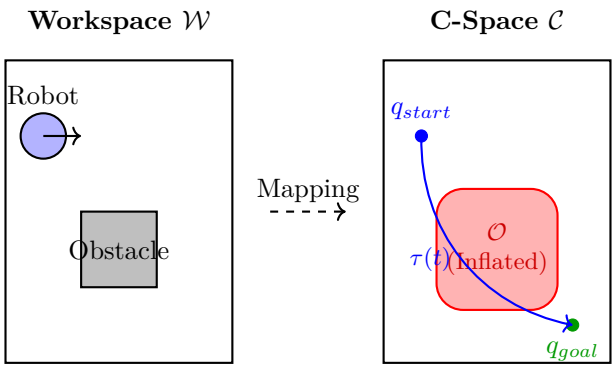


Figure 7: From Workspace to C-Space: to treat the robot as a mathematical point q (facilitating path search τ), obstacles are expanded by the robot's radius to form the collision region $\mathcal{O}$.

The formal problem of planning thus becomes searching for a continuous function $\tau : [0, 1] \to \mathcal{C}_{free}$, such that $\tau(0) = q_{start}$ and $\tau(1) = q_{goal}$.

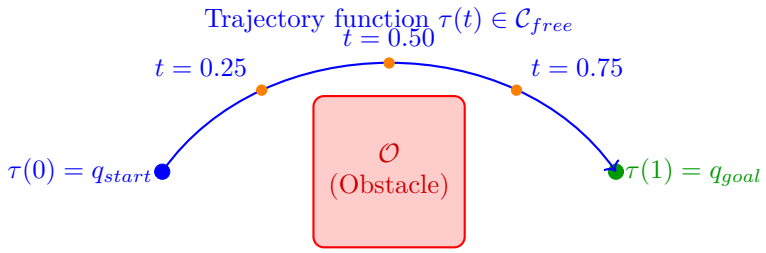


Figure 8: Geometric representation of the function $\tau : [0, 1] \to \mathcal{C}_{free}$. The progression of parameter t from 0 to 1 "draws" the continuous path from start to goal, bypassing collision zones.

During this search, the system is subject to two types of **kinematic constraints**:

1. **Holonomic Constraints:** Equations of the form $g(q) = 0$ that limit reachable positions; furthermore, each independent holonomic constraint reduces the system's degrees of freedom by 1. Consider a roller coaster car. Although living in a three-dimensional space ($\mathbb{R}^3$), the car is constrained to slide along a fixed physical guide (the track). The equations describing the track are of the form $g(q) = 0$. Due to this constraint, the car's position and orientation are determined by a single variable: the distance traveled along the track. The system has lost its original Degrees of Freedom, reducing to just 1 DOF. The robot *cannot* reach points outside the track.

2. **Non-Holonomic Constraints:** Differential equations of the form $u(q)\dot{q} = 0$ that restrict the —velocities— and directions of motion without reducing DOFs. The classic example is a car: it can reach any coordinate (x, y) in a parking lot (preserving 3 DOFs), but cannot move sideways (derivative constraint). To better understand, the lateral component of velocity must be zero ($v_y = 0$ in the local reference frame). Consequently, to move sideways, the car cannot travel in a straight line but must execute a complex maneuver (like an S-shaped parallel parking maneuver).

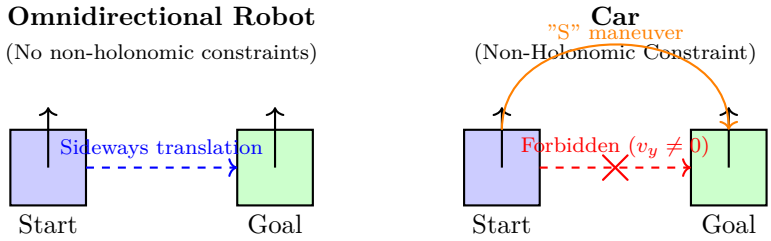


Figure 9: Practical difference in navigation. An omnidirectional robot can slide sideways. A car (subject to non-holonomic constraints on its wheels) cannot skid laterally, forcing the *Local Planner* to compute more complex curved trajectories to reach the same goal.

Local Planners for Trajectory Execution

To navigate while respecting kinematics and non-holonomic constraints, and to react to unforeseen dynamic obstacles, high-frequency real-time *Local Planning* algorithms are used:

- **Dynamic Window Approach (DWA):** Instead of calculating a geometric path, DWA works directly in the *velocity space* (translational v and rotational ω). At each instant, it samples a "window" of reachable velocities for the robot in the next *time-step* (given its maximum accelerations). For each pair (v, ω), it simulates a short circular trajectory and evaluates it via an objective function that rewards progress toward the goal and penalizes proximity to obstacles. It is computationally lightweight and perfect for differential robots on grids, but tends to get trapped in local minima in highly confined environments (e.g., U-shaped corridors).
- **Timed Elastic Bands (TEB):** Formulated as a multi-objective non-linear optimization problem in the space-time domain. The algorithm initializes a crude trajectory between start and goal. This trajectory is modeled as a series of points connected by "virtual elastic springs". Attractive virtual forces pull the trajectory toward the goal and maintain its fluidity (respecting kinematics), while repulsive forces (generated by obstacles) "deform" the elastic band to maintain a safety distance. By adjusting temporal weights, TEB guarantees that the robot reaches the goal in the shortest time possible without violating velocity or acceleration bounds.

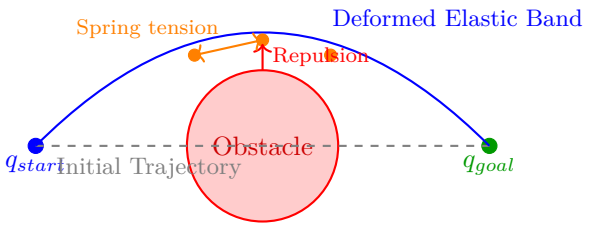


Figure 10: Concept of the TEB (*Timed Elastic Band*) Local Planner: The trajectory is modeled as an elastic band deformed by repulsive forces from the obstacle, maintaining kinematic consistency via internal tension.

7 Sampling-Based Motion Planning (SBMP)

When a robot possesses many degrees of freedom (for example, a robotic arm with 6 or 7 joints), its $\mathcal{C}$-Space becomes a 6 or 7-dimensional space. In high-dimensional spaces, explicitly constructing or discretizing the entire

space to map obstacles (creating an exact *Roadmap*) becomes computationally intractable due to the "curse of dimensionality".

Sampling-Based Motion Planning (SBMP) techniques bypass this problem by renouncing full mapping of the space. Instead, they explore $\mathcal{C}_{free}$ "blindly" via probabilistic sampling: they generate random configurations and verify whether they are valid (collision-free). Although they lose absolute guarantees of finding the optimal path (—optimality—), they offer a highly powerful property: **Probabilistic Completeness**. If a solution exists, the probability that the algorithm finds it approaches 1 as execution time goes to infinity.

The two fundamental algorithms of this family are PRM and RRT.

7.1 Probabilistic Roadmaps (PRM)

PRM is a **multi-query** algorithm, designed for static environments where the map does not change and where multiple planning requests will be made over time. It is split into two distinct phases:

1. **Learning Phase (Offline Construction):** The algorithm randomly samples a large number of points in $\mathcal{C}$-Space. It discards those falling into obstacles ($\mathcal{O}$) and keeps those in $\mathcal{C}_{free}$. Subsequently, it attempts to connect each node to its k nearest neighbors using a simple and fast —Local Planner— (such as a straight interpolated line). If the straight line segment does not intersect any obstacles, the edge is saved. The result is a graph (the *Roadmap*) capturing the topology of the free space.

2. **Query Phase (Online Search):** When a task is assigned, the start node q_{start} and the goal q_{goal} are temporarily hooked to the nearest nodes of the Roadmap. At this point, the problem reduces to a graph search, instantly resolvable using standard algorithms like A^* or Dijkstra.

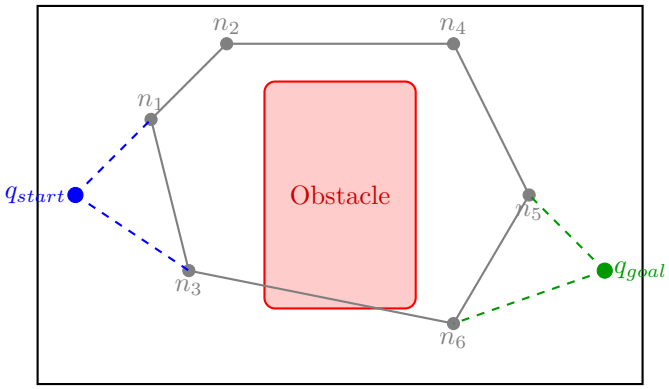


Figure 11: The PRM algorithm: the gray network is the Roadmap generated offline during the Learning phase. Dashed lines illustrate the Query phase, where q_{start} and q_{goal} connect to the graph to find a path.

7.2 Rapidly-exploring Random Trees (RRT)

Unlike PRM, RRT is a **single-query** algorithm: it does not construct a global map for reuse, but builds a specific tree structure from scratch to solve the single current problem. It is ideal for dynamic or very high-dimensional environments.

The algorithm grows a tree rooted at q_{start} by iteratively repeating three steps:

1. **Sampling:** Generates a random point q_{rand} in the space.

2. **Nearest Neighbor Search:** Finds the node $q_{nearest}$ currently in the tree that is closest to the point q_{rand}.

3. **Extension (Steering):** The tree does not snap directly to q_{rand}, but takes "a step" in that direction (a calculated configuration constraint, not an actual step). It creates a new node q_{new} by advancing from $q_{nearest}$ toward q_{rand} by a preset fixed distance Δq. If the segment between $q_{nearest}$ and q_{new} does not cross any obstacles, the node is added to the tree.

The Secret of RRT: The Voronoi Bias

What makes RRT incredibly efficient at rapidly exploring complex spaces is its inherent **Voronoi Bias**. If the space is partitioned into the Voronoi Cells of the tree (where each cell represents the zone of influence of a node), the "frontier" nodes (those on the outer edges of the tree) possess cells with a much larger surface area compared to nodes trapped in the center.

Since q_{rand} is sampled with uniform probability across the entire space, it is statistically far more likely that a sample falls into the huge cell of a frontier node. This constantly "pulls" the tree growth toward unexplored regions, preventing it from unnecessarily clustering around the starting point.

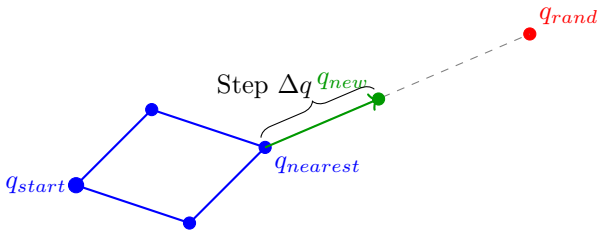


Figure 12: —
Expansion in RRT: once a random point q_{rand} is sampled, the nearest node $q_{nearest}$ is identified, and the tree is extended by a fixed step Δq to generate q_{new}.—